```
import numpy as np

# --- SYSTEM SIMULATION ENVIRONMENT ---
def evaluate_fitness(position, solar_data, wind_data, load_data):
    """
    Evaluates the total cost + reliability penalty of a specific component mix.
    position[0] = Number of PV panels
    position[1] = Number of Wind Turbines
    position[2] = Number of Battery Units
    """
    n_pv, n_wind, n_batt = position

    # Simple simulated yearly outputs (normally an hourly loop over 8760 hours)
    total_gen = (n_pv * np.sum(solar_data)) + (n_wind * np.sum(wind_data))
    total_load = np.sum(load_data)

    # Capital Cost calculation (Hypothetical asset unit costs)
    cap_cost = (n_pv * 150) + (n_wind * 1200) + (n_batt * 300)

    # Reliability check (LPSP)
    unmet_energy = max(0, total_load - (total_gen + n_batt * 50))
    penalty = unmet_energy * 1000  # High penalty factor for blackouts

    return cap_cost + penalty

# --- PSO PARAMETERS ---
num_particles = 30
max_iter = 100
dimensions = 3  # PV, Wind, Battery
bounds_min = np.array([0, 0, 0])
bounds_max = np.array([200, 50, 100])

w, c1, c2 = 0.7, 1.5, 1.5

# Dummy annual aggregated tracking data
solar_profile = np.random.uniform(0.1, 0.5, 100)
wind_profile = np.random.uniform(0.2, 0.8, 100)
load_profile = np.random.uniform(20, 100, 100)

# --- INITIALIZATION ---
X = np.random.uniform(bounds_min, bounds_max, (num_particles, dimensions))
V = np.random.uniform(-1, 1, (num_particles, dimensions))

P_best = np.copy(X)
P_best_fitness = np.array([evaluate_fitness(p, solar_profile, wind_profile, load_profile) for p in X])

g_best_index = np.argmin(P_best_fitness)
G_best = np.copy(P_best[g_best_index])
G_best_fitness = P_best_fitness[g_best_index]

# --- OPTIMIZATION LOOP ---
for iteration in range(max_iter):
    for i in range(num_particles):
        # 1. Update Velocity
        r1, r2 = np.random.rand(dimensions), np.random.rand(dimensions)
        V[i] = (w * V[i] +
                c1 * r1 * (P_best[i] - X[i]) +
                c2 * r2 * (G_best - X[i]))

        # 2. Update Position
        X[i] = X[i] + V[i]

        # 3. Apply Boundary Constraints
        X[i] = np.clip(X[i], bounds_min, bounds_max)

        # 4. Evaluate Fitness
        fitness = evaluate_fitness(X[i], solar_profile, wind_profile, load_profile)

        # 5. Update Personal Best
        if fitness < P_best_fitness[i]:
            P_best[i] = np.copy(X[i])
            P_best_fitness[i] = fitness

            # 6. Update Global Best
            if fitness < G_best_fitness:
                G_best = np.copy(X[i])
                G_best_fitness = fitness

print(f"Optimization Finished.")
print(f"Optimal Configuration -> PV Panels: {round(G_best[0])}, Wind Turbines: {round(G_best[1])}, Batteries: {round(G_best[
print(f"Minimized System Metric Value: ${G_best_fitness:,.2f}")
```

```
print(f"Minimized System Metric Value: ${g_best_fitness:.2f}")
```

```
Optimization Finished.
Optimal Configuration -> PV Panels: 200, Wind Turbines: 0, Batteries: 8
Minimized System Metric Value: $32418.87
```